

EE 446: TinyML

Assignment 2: Programming Results Writeup

Student Information

Full Name: Luke Valerio

Student ID: 1901711

Repository: https://github.com/LukeVal27/tiny_ML

Assignment 2 has no questions marked [Dis]. All five problems are [Pro] and they add up to 100 points. The handout still asks for a PDF named *FirstName_LastName_EE446_HW2_Dis.pdf*, and problems 3(c), 4(b), 5(c) and 5(d) each ask for results to be reported in the writeup with screenshots, so that is what this file holds. Every number below comes from *Luke_Valerio_EE446_HW2_Pro.ipynb*, run start to finish with `random_state=42` so the train/test split stays the same on every run.

Question 3(c): float32 base_model on the test set

The network is four fully connected layers, 38 to 64 to 32 to 16 to 1, with ReLU on the hidden layers and one sigmoid output neuron. Training ran for 20 epochs using Adam and binary cross entropy. I kept the model small on purpose. The Arduino sketch allocates a 32 KB tensor arena, and the weights plus every intermediate activation have to fit inside it.

Q3(c) FLOAT32 base_model - test set performance

Accuracy: 0.9985

	precision	recall	f1-score	support
attack (0)	0.9984	0.9985	0.9984	11726
normal (1)	0.9987	0.9986	0.9986	13469
accuracy			0.9985	25195
macro avg	0.9985	0.9985	0.9985	25195
weighted avg	0.9985	0.9985	0.9985	25195

Confusion matrix (rows = actual, cols = predicted):

```
[[11708    18]
 [    19 13450]]
```

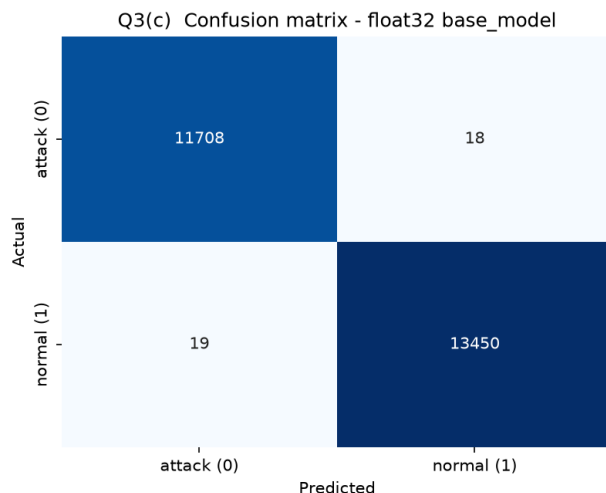


Figure 1. Confusion matrix, float32 base_model, 25,195 test samples.

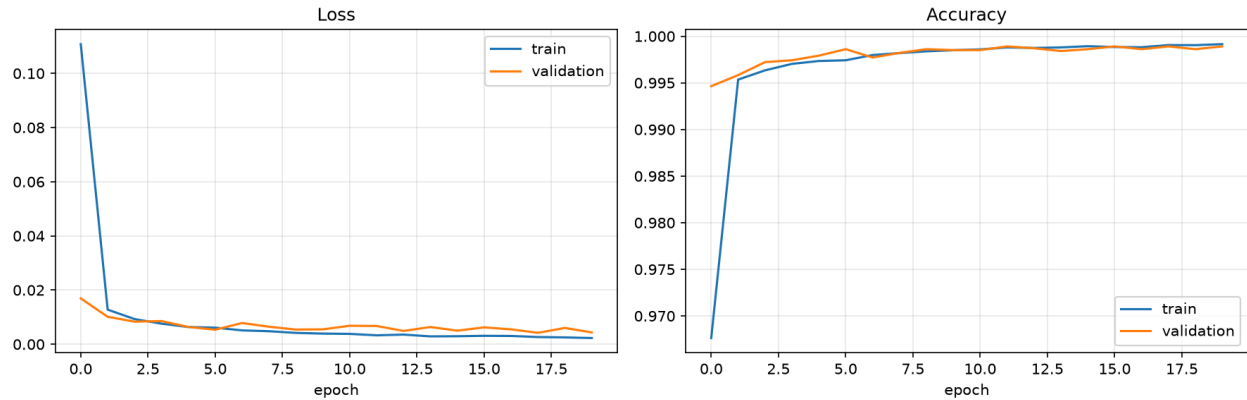


Figure 2. Training and validation loss and accuracy over 20 epochs.

Question 4(b): INT8 tfliite_quant_model on the test set

Post-training quantization used a 500 sample representative dataset drawn from X_train to calibrate the activation ranges. The converter was restricted to TFLITE_BUILTINS_INT8 operators, with inference_input_type and inference_output_type both set to tf.int8, so nothing in the deployed graph runs in floating point. Each test sample gets quantized with the input tensor's own scale and zero point before inference, and the INT8 result is dequantized with the output tensor's scale and zero point.

Q4(b) INT8 tfliite_quant_model - test set performance

input scale = 0.0895969868 zero_point = -52
output scale = 0.0039062500 zero_point = -128

Accuracy: 0.9984

	precision	recall	f1-score	support
attack (0)	0.9979	0.9987	0.9983	11726
normal (1)	0.9989	0.9981	0.9985	13469
accuracy			0.9984	25195
macro avg	0.9984	0.9984	0.9984	25195
weighted avg	0.9984	0.9984	0.9984	25195

Confusion matrix (rows = actual, cols = predicted):

```
[[11711  15]
 [ 25 13444]]
```

float32 accuracy : 0.9985
INT8 accuracy : 0.9984
change : -0.0001

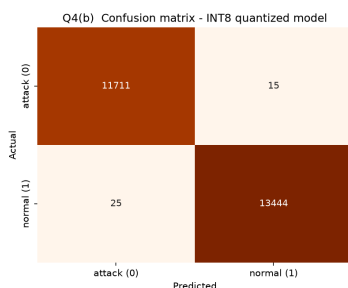


Figure 3. Confusion matrix, INT8 quantized model, 25,195 test samples.

Float32 against INT8

Model	Size	Test accuracy
float32 Keras (original_model.h5)	100.01 KB	0.9985
INT8 TFLite (quantized_model.tflite)	8.24 KB	0.9984
Change	12.1× smaller	−0.0001

Quantization cost one hundredth of a percentage point of accuracy. That is three extra misclassifications out of 25,195, in exchange for a model more than twelve times smaller. Without that trade the model does not fit on the Nano 33 BLE at all.

Question 5(c): Serial Monitor output, 5 samples

Sketch: network_data/network_data.ino, holding rows 0 to 4 of X_test. The INT8 output tensor is dequantized as $(\text{value} - \text{zero_point}) \times \text{scale}$, then thresholded at 0.5. The quantization parameters baked into the model are input scale 0.0895969868 with zero point −52, and output scale 0.0039062500 with zero point −128.

Output	Serial Monitor	×
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem101')		
Sample #1	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #2	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #3	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #4	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #0	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #1	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #2	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #3	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #4	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #0	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #1	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #2	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #3	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #4	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #0	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #1	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #2	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #3	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)

Figure 4. Serial Monitor output on the Arduino Nano 33 BLE for the five test samples. `loop()` runs forever, so several identical cycles of Sample #0 through #4 are visible. All five predictions match the actual labels and reproduce the desktop TFLite interpreter exactly.

Question 5(d): Serial Monitor output, 10 samples

Sketch: network_data_10/network_data_10.ino, holding rows 5 to 14 of X_test. These are the ten samples that follow the five used in part (c), as the handout asks.

```
Output  Serial Monitor ×
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem101')

Sample #0 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #1 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #2 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #3 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #4 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #5 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #6 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #7 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #8 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #9 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #0 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #1 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #2 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #3 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #4 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #5 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #6 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #7 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #8 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #9 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
```

Figure 5. Serial Monitor output for the ten test samples, two complete cycles shown. All ten predictions match the actual labels: six attack and four normal.

Every sigmoid value reads either 0.0000 or 0.9961 and never anything between the two. That is the INT8 output tensor saturating rather than a defect. Output scale is 1/256 with a zero point of -128, so an INT8 value of -128 dequantizes to exactly 0.0 and the maximum of +127 dequantizes to 255/256, or 0.99609375. The model is confident enough on all fifteen samples that it pins to one rail or the other, which leaves the quantized sigmoid no way to report a value in between.

Expected on-device predictions

These come from the desktop TFLite interpreter running the same quantized model, so the Serial Monitor output can be checked against a known reference. All fifteen samples are classified correctly.

```
Expected predictions from the desktop INT8 interpreter
input  scale=0.0895969868 zero_point=-52
output scale=0.0039062500 zero_point=-128

Q5(c) samples 0-4
Sample 0: prob=0.0000 predicted=0 actual=0
Sample 1: prob=0.9961 predicted=1 actual=1
Sample 2: prob=0.9961 predicted=1 actual=1
Sample 3: prob=0.0000 predicted=0 actual=0
Sample 4: prob=0.0000 predicted=0 actual=0

Q5(d) samples 5-14
Sample 5: prob=0.0000 predicted=0 actual=0
Sample 6: prob=0.9961 predicted=1 actual=1
```

```
Sample 7: prob=0.0000 predicted=0 actual=0
Sample 8: prob=0.9961 predicted=1 actual=1
Sample 9: prob=0.0000 predicted=0 actual=0
Sample 10: prob=0.0000 predicted=0 actual=0
Sample 11: prob=0.0000 predicted=0 actual=0
Sample 12: prob=0.9961 predicted=1 actual=1
Sample 13: prob=0.0000 predicted=0 actual=0
Sample 14: prob=0.9961 predicted=1 actual=1
```